Scheduling Trust: Latency-Sensitive Serverless Computing on Attested Volunteer Machines

Aalhad Sawane

***Abstract*—Idle consumer machines could serve short serverless invocations close to users, but their owners are unreliable and possibly malicious. We argue that remote attestation of a locked appliance image reduces a malicious owner to an omission or timing fault, and that what remains is a scheduling problem in which *trust is a perishable resource*: costly to establish, destroyed by churn, and exposing every host that holds a tenant's key. We present VOSS, a two-timescale scheduler. A slow loop solves a robust mixed-integer program that decides where trust and warm state live, covering demand against *any* f defecting hosts. A fast loop places each invocation with a cost model and hedges risky ones. Progress-based failure detection, scheduler succession without volunteer consensus and attested receipts complete the design. We evaluate VOSS in a deterministic simulator, calibrated with measurements from a real TPM-equipped machine (reboot to kernel 21 s, quote 0.51 s), over 21 experiments with 10 seeds each. With a sync P99 target of 3 s, VOSS is viable in 9 of 9 and 13 of 16 cells of two regime maps in which the strongest simple baseline never is. Against no planning the planner cuts sync P99 by 55–60% and against keep-alive by 29–58%, at about 2.1× the warm memory; progress detection holds success at 99.95% under 20% omitting owners where heartbeat-only detection drops to 96.8%. Several expected benefits do not materialize: geo-clustering is 5–17% slower than one global scheduler at our scale, reputation tie-breaking changes P99 by only 2.4%, VOSS's P99 is 14% above that of a commercial-style reference, and success degrades beyond 3×10^4 workers. All hypotheses, including the 5 refuted ones, are reported with paired confidence intervals.**

***Index Terms*—Serverless computing, volunteer computing, remote attestation, trusted execution, scheduling, robust optimization, tail latency, simulation.**

I. INTRODUCTION

Most serverless invocations are short: in production traces a large share of functions run for well under a second [1], and for such functions network distance and cold starts, not computation, dominate latency. Idle consumer machines are abundant, close to users, and owned by nobody who has to be paid in capital expenditure. Volunteer computing has harvested them for two decades [2], [3], but only for delay-tolerant batch work whose deadlines are hours or days. Serving *latency-sensitive* functions from volunteer hardware raises two problems that batch systems avoid. The machines are *unreliable*: owners reboot, unplug and reclaim them at will, so every warm sandbox is perishable. And they are *untrusted*: an owner can read a tenant's code and data, return forged results, or defect at the worst moment.

Classical volunteer systems answer the second problem with redundancy and voting, which triples cost and does not apply to functions with side effects. Our starting point is different. If tenant code and the scheduler's own code run only inside a measured, owner-locked appliance image whose TPM-backed attestation a verifier checks, a hostile owner can no longer read, forge or tamper; what remains is *omission and timing*: power off, drop, delay, throttle. Attestation thus reduces Byzantine owners to the fault class that churn already forces a volunteer system to tolerate [4], [5]. We make this reduction the foundation of the design rather than a footnote.

The reduction changes what scheduling is about. *Trust* becomes a resource with three properties. It is costly to establish: booting into the appliance, attesting, and releasing a tenant's key to a host all take time. It is perishable: all of it vanishes when the owner ends the donation session. And it is exposing: every additional host that holds a tenant's key widens that tenant's attack surface. Placing an invocation on a host that is not yet trusted for the tenant is therefore not a free choice, and neither is asking more hosts to hold the key.

We present VOSS (volunteer-oriented serverless scheduling), a scheduler built around this view. Our contributions are:

1) **A threat-reduction argument and substrate model** (Sec. III): attestation maps malicious owners to omission/timing faults, and a data-driven substrate profile (TPM appliance, confidential VMs, DRTM, unattested) makes trust establishment, perishability, warm-state ladders and assurance explicit.
2) **Two-timescale, trust-aware scheduling** (Sec. IV): a slow robust mixed-integer program plans *where trust and warm state live*, with coverage that survives any f defecting hosts, memory and exposure costs and failure-domain caps; a fast placer decides *where each invocation runs* from a cost model over the same plan, with risk-proportional hedging.
3) **Fault tolerance that treats liveness as a hint** (Sec. IV): progress-based detection, scheduler succession without volunteer consensus, and attested receipts that make accounting verifiable without a blockchain.
4) **A deterministic, calibrated simulator and a mechanical evaluation** (Sec. V): paired comparisons over 10 seeds, a viability map over churn, adversaries, substrates, load and population, a latency–exposure–robustness Pareto analysis, and a scorecard in which each hypothesis has a stated verdict rule, so that regimes where baselines win are reported by construction.
5) **Measurements** from a TPM-equipped machine that parameterize attestation, boot and snapshot costs (Sec. V).

Our results (Sec. VI) are mixed, and we report them as such. Planning trust and warm state is the largest lever: removing it costs 1.1 s of sync P99, and removing lifecycle-aware placement

[TODO: author list, affiliations, funding and acknowledgments; this draft names only the repository owner.]

costs 3.3 s. Robust planning buys tail latency at the price of key exposure. Progress-based detection is decisive against omitting owners and black-holed schedulers. But geo-clustering, a design pillar of earlier volunteer-scheduling proposals, *loses* to one global scheduler at our scale; reputation, tiered caches and the integer program over a greedy planner each give little or nothing measurable; a simple baseline holds about half of VOSS's key exposure; a commercial-style reference cloud has lower P99; and the system degrades at 81% success at 10^5 workers.

Scope and honesty. All results come from simulation. The simulator is calibrated with measurements from one TPM-equipped consumer laptop, not from the target node, and it has not been validated end to end against a real multi-machine testbed (Sec. VII-A). Confidential-VM substrates use literature parameters. We therefore read our numbers as evidence about *mechanisms and regimes*, not as performance predictions.

The paper is organized as follows. Section II positions the work. Sections III and IV describe the model and the design. Section V describes the simulator, calibration and protocol, Section VI the results, and Sections VII–VIII discuss limits and conclude.

II. Background and Related Work

Volunteer and desktop-grid computing. BOINC and SETI@home showed that idle machines can deliver large throughput [2], [3], and availability studies characterize heavy-tailed, time-of-day dependent session lengths [6]. These systems target batch work with generous deadlines and defend against cheating with replication and validation. VOSS borrows the availability model but targets sub-second invocations, and replaces replication with attestation.

Serverless scheduling and cold starts. Production characterizations show skewed popularity and short executions [1]; microVMs such as Firecracker [7] and snapshot-based restore [8], [9], container provisioning [10] and keep-alive caching policies [11] attack the cold start. All assume provider-owned, reliable machines. Work on harvested resources [12] tolerates evictions inside one provider; edge serverless schedulers handle heterogeneity and locality [13]. We reuse the lifecycle ladder (artifacts, snapshot, warm VM) and a GreedyDual-style value function for eviction [14], and add what volunteers need on top: perishable trust and an explicit exposure cost. Our workload kernels follow SeBS [15].

Confidential serverless computing. Confidential VMs (AMD SEV-SNP [16], Intel TDX) protect tenants from the host, at a price: boot is more than twice a plain VM [17], snapshot and templating techniques do not transfer because memory is per-VM encrypted, and network I/O pays a bounce-buffer tax [18]. Systems such as SEVeriFast [19], serverless confidential containers [20] and trustlet-based designs [21] attack these costs, but all need server-class CPUs. Consumer hardware offers only TPM-based measured boot and, at best, memory encryption that is not attestable; physical attacks on fTPMs exist [22], which we exclude from the threat model. VOSS is substrate-agnostic: the TPM appliance is the primary profile, and CVM classes enter as parameterized profiles whose numbers come from this literature.

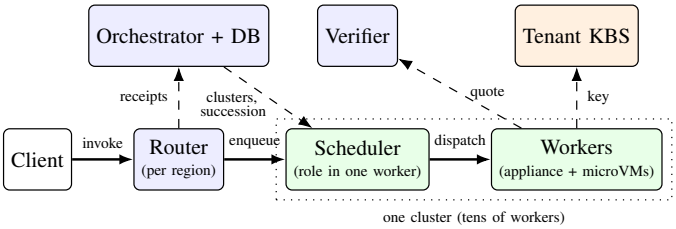


Fig. 1. Roles. Blue: provider-run orchestrator tier (trusted for availability and accounting only). Green: volunteer machines running the attested appliance (their *hosts* may still omit or delay). Orange: tenant-operated key broker. Solid arrows: the invocation path; dashed: control and trust.

Attestation and trusted hardware in distributed systems. Keylime-style architectures combine measured boot, IMA and key release [23]. Trusted counters and logs turn Byzantine behavior into omission [4], [5], which is the fault model VOSS adopts; rollback of persisted state is the known caveat, and our schedulers therefore keep only soft state. We do not use a blockchain: receipts signed by attested session keys already make work third-party verifiable, and the trusted party could only withhold records.

Tail latency, hierarchy and optimization. Hedged and tied requests are the standard defense against stragglers [24]; we make hedging risk-proportional and place the copy in another failure domain. Sparrow-style sampling [25] and Borg-style cells [26] motivate hierarchical, partitioned scheduling; our clusters are geographic, which Sec. VI shows does not pay at our scale. The planner's robust coverage constraint is the budgeted-uncertainty construction of Bertsimas and Sim [27], written with the dual form of the sum of the f largest terms; we solve it with HiGHS [28]. Reputation systems [29] and Sybil resistance [30] inform our identity model (one TPM, one identity). Failure detection builds on accrual detectors [31], but treats heartbeats as hints and progress as evidence.

III. System and Threat Model

A. Roles

Figure 1 shows the roles. A provider-run *orchestrator tier* (orchestrator, regional routers, operational database, attestation verifier) is trusted for availability and accounting but never sees tenant secrets. Each tenant runs its own *key broker* (KBS). *Workers* are volunteer machines booted into the attested VOSS appliance, which runs tenant functions in Firecracker microVMs. One worker per cluster additionally plays the *scheduler* role; every attested worker carries the scheduler code in the same image, so promoting a successor costs no boot. The orchestrator partitions workers into small geographic clusters (tens of workers), which bounds the planner's problem size and intra-cluster latency.

B. Donation sessions and attestation

A worker alternates between OFF and *sessions*. A session starts when the owner reboots into the appliance (scheduled, idle-triggered, or on a dedicated spare box), becomes schedulable when attestation completes, and ends abruptly (owner reboot, power loss) or gracefully (a drain request with a

deadline). Session lengths are heavy-tailed and time-of-day dependent, with regional shocks. The appliance image is a signed unified kernel image with a dm-verity root, no owner login, and a data partition sealed to the expected PCR values; the disk caches survive a session as ciphertext.

Attestation follows the Keylime pattern [23]. The agent presents the TPM's endorsement-key certificate, then a quote over PCRs and the IMA log bound to a nonce and a freshly generated session public key; the verifier replays the event log against a policy and returns a session certificate that expires unless re-quoted. All channels use mutual TLS under that certificate, so no untrusted OS lies on any path. A tenant's KBS releases that tenant's key, wrapped to the session key, only to such a host. *Trust-warm* means that the key is present in appliance memory this session; it is lost with the session. The scheduler and orchestrator never see tenant keys.

C. Threat model

Volunteer owners control power, network, firmware settings and their own OS, may try to boot modified software, run the appliance in a VM, replay, delay or drop traffic, create identities, and choose when to defect. Measured boot, a vendor-certified TPM endorsement key, an owner-locked image and sealed storage defeat the first group; what remains is *omission and timing*: power off, drop, delay, throttle, and lying about anything outside the attested stack (including fake drain notices). *Network attackers* see only ciphertext. *Other tenants* are contained by microVM isolation. Out of scope: lab-grade physical attacks (for example fTPM voltage glitching [22]), kernel and VMM zero-days in the appliance, side channels and traffic analysis, and failure of or attacks on the orchestrator tier. Identity is the hash of the TPM endorsement key, so identities cost machines [30].

Consequence. To the rest of the system a malicious worker or scheduler host is an omission/timing-faulty node. The same mechanisms therefore cover churn, crashes and malice (Sec. IV).

D. Substrate profiles

A *substrate profile* is data that tells one generic lifecycle engine how a worker establishes trust, how long it lasts, what the warm-state ladder looks like, and what assurance a host of that class carries. We model five. `appliance_tpm` (primary) pays a reboot, measured boot and a TPM quote once per session. `cvm_snp` and `coco_pool` model server confidential VMs and confidential containers with per-wCVM or per-shell establishment and trustlet-style warm levels, with parameters from the literature (ranges swept in experiments). `drtm_hv` models a measured hypervisor launched under the owner's OS (no reboot, owner interference). `none` is the unattested reference.

For appliance and unattested hosts the ladder has five levels per (worker, function): L0 nothing cached; L1 code bundle and root file system on disk; L2 snapshot on disk; L3 warm idle VM in RAM; L4 busy. Trust is an orthogonal flag $K[t, j]$.

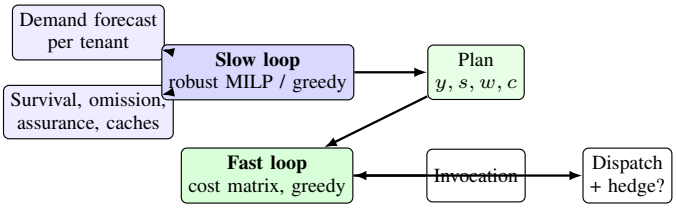


Fig. 2. Two timescales. The planner decides where trust and warm state live; the placer decides where each invocation runs, over candidates and earmarks the plan provides.

E. Exposure

Let $a_j \in [0, 1]$ be the assurance of host j (substrate base, freshness of the last quote, firmware policy), and $\mathrm{KH}(t)$ the set of hosts holding tenant t's key or decrypted artifacts. The *exposure* of t is $E_t = \sum_{j \in \mathrm{KH}(t)} (1 - a_j)$, weighted by the tenant's sensitivity class σ_t in the planner. It is a transparent proxy for the attack surface, not a probability of compromise, and we therefore report latency, exposure and robustness side by side rather than as one weighted number.

IV. The Voss Design

A. Why two timescales

Trust and warm state are set-valued and long-lived: a key released to a host stays for the session; exposure depends on *how many distinct hosts* hold a key; and robustness to f defectors is a worst case over subsets. None of these is a per-assignment additive cost, so none belongs in a per-request optimization. Placement, in contrast, is latency-critical and decomposes well. Voss therefore splits the decision (Fig. 2): a *slow loop* (the trust and warm-state planner, every 10–60 s and on events) decides which hosts are trust-warm for which tenants, which snapshots and warm VMs to keep, and how many slots to earmark; a *fast loop* places each invocation using that plan.

B. Slow loop: trust and warm-state planning

For one cluster with attested workers J and *hot* tenants and functions (those above a rate threshold; the long tail uses a cold path), the planner chooses $y[t, j] \in \{0, 1\}$ (tenant t is trust-warm on j), $s[f, j] \in \{0, 1\}$ (snapshot of f pinned on j), $w[f, j] \in \mathbb{Z}_{\geq 0}$ (warm VMs of f on j), and $c[t, j] \geq 0$ (slots of j earmarked for t). Inputs are estimates only: slots cap_j, the survival probability $S_j(H)$ over the horizon, the omission rate p_j, assurance a_j, failure domain $d(j)$, the demand quantile q_t (a Poisson quantile of the forecast arrival rate times service time), sensitivity σ_t, assurance floor $a_{\min,t}$ and robustness level f_t. The objective (minimized; terms are logged separately) is

$$\begin{aligned} \min \; & \sum_t C_t u_t + \sum_j \Big(\rho_{\mathrm{mem}} \sum_f m_f w[f, j] + \rho_{\mathrm{ssd}} \sum_f z_f s[f, j] \Big) \\ & + \sum_{t,j} \sigma_t (1 - a_j)\, y[t, j] + \kappa \sum_{t,j} (y[t, j] - y^{\mathrm{prev}}[t, j])^+ \\ & - \sum_{f,j} \beta\, b_f\, (w[f, j] + \gamma s[f, j]), \end{aligned} \tag{1}$$

that is: uncovered demand u_t priced at the cold penalty C_t, donor memory and storage, exposure, plan churn, and the start-up time saved by warm VMs and snapshots (memory m_f, snapshot size z_f, benefit b_f). The central constraint is robust coverage: with at most f_t of the earmarked hosts defecting,

$$\sum_j c[t,j]\, S_j(H)(1-p_j) \;-\; \big(f_t\,\theta_t + \textstyle\sum_j \pi[t,j]\big) + u_t \;\geq\; q_t,$$
$$\pi[t,j] \geq c[t,j] - \theta_t, \quad \pi[t,j] \geq 0, \tag{2}$$

where $f_t\theta_t + \sum_j \pi[t,j]$ equals the sum of the f_t largest contributions by LP duality, the construction of [27]. Because the constraint holds against *any* f_t hosts, including ones with a perfect record, robustness does not depend on the omission model. Further constraints couple the variables ($c \leq \mathrm{cap}\, y$, $w \leq W_{\max}\, y$, $s \leq y$), bound slots, memory and SSD per host, cap the share of earmarked capacity in any failure domain by κ_{dom}, forbid $y[t,j] = 1$ when $a_j < a_{\min,t}$ or the attestation expires within the horizon, and the default $f_t = \lceil \alpha_{\mathrm{assumed}} N_{\mathrm{cluster}} \rceil$. A cluster has about 30 workers, at most 20 hot tenants and 50 hot functions, so the program has 10^3–$3\cdot10^3$ integers; we solve it with HiGHS [28] within a time limit and fall back to the incumbent and then to a greedy planner that adds workers by value density until the robust coverage holds. The plan is diffed against its predecessor into idempotent actions (PREPARE, PIN-SNAPSHOT, WARM, RELEASE), rate-limited per worker; the fast loop only relies on state workers have *confirmed*.

C. Fast loop: placement, hedging and routing

For an invocation of tenant t, candidates are the confirmed trust-warm workers (with earmarked slots) and, behind them, other attested workers meeting the assurance floor, which pay a key-release round trip and an exposure increment. Each pair (i,j) gets a completion distribution (a moment-matched shifted lognormal over wait, trust, start-up by ladder level, network and execution) and a failure probability $p_{\mathrm{fail}} = 1 - S_j(T)(1-p_j)$. The default scalarization is $w_i\, p_{\mathrm{miss}} + \epsilon\, \mathbb{E}[T] + \lambda_{\mathrm{exp}}\, \mathrm{exposure}$. Execution time comes from a Bayesian log-linear predictor with hierarchical shrinkage and conformal calibration [32]. If p_{miss} of the best candidate exceeds a threshold θ and a per-cluster budget allows, VOSS sends a *tied* duplicate to the best trust-warm candidate in a *different failure domain* and cancels the loser on the first result [24]; attested results make duplicates integrity-safe. Routers pick the cluster with a score $\mathrm{rtt} + \mathbb{E}[\mathrm{completion}] + \mathrm{health} + \mathrm{trust_cold_penalty}$, so spilling to a cluster without trust-warm capacity is a priced decision.

D. Fault tolerance: liveness is a hint, progress is evidence

A black-holing owner keeps heartbeating, so heartbeats cannot be evidence. VOSS detects *progress*: routers (trusted) time ENQUEUE without ACCEPTED; schedulers time ACCEPTED without RESULT at the predicted completion quantile; the orchestrator watches the scheduler's attested placement latency; and a CUSUM change-point test over outcomes defeats on-off defection. A suspect worker loses new work and trust-warm eligibility and its in-flight work is hedged or retried; fake drain notices only make the scheduler more conservative and never excuse an omission.

Succession without consensus. Because the orchestrator is a reliable arbiter, a failed scheduler is replaced by an atomic compare-and-swap on the cluster epoch in the operational database; the appointment token binds the cluster, epoch and the *session-certificate* of the successor, so a cloned image is not appointed, and workers fence lower epochs. Schedulers keep only soft state: undispatched invocations live in the router, running work in the workers (recovered by a roll call), and plan and learned state in a checkpoint. A restarted or rolled-back appliance is a new instance and must be re-appointed, which closes the rollback caveat of trusted-hardware protocols.

E. Receipts and credits

Each result carries a receipt signed by the worker's session key (invocation, attempt, worker, certificate, epoch, execution time, reference work units, output digest). A host cannot claim work it did not do, only fail to do work. Credits are paid in *reference work units*, not elapsed time, so throttling earns nothing; donor caching earns a small credit only while the worker answers probes. Reputation, an identity-level long-run signal, feeds rewards and planner tie-breaks but never the failure probability p_{fail}, which is fed separately by session survival and omission, to avoid double counting.

V. Evaluation Method

A. Simulator

All results come from a deterministic discrete-event simulator that runs the *same policy code* as the scheduler. Policies (planner, placer, router, hedger, detector) are pure functions of immutable views; the layered code base enforces by static import checks that a policy cannot import the simulated world, so it cannot read future state. Every cross-node interaction is a typed message delivered after a delay derived from great-circle distance between about 100 metro areas plus a per-hop overhead. Nothing is free: network transfer, disk-tier reads, CPU contention, TPM quotes, verification, key release and the decision latency of planner, placer, router and orchestrator all consume simulated time. All randomness comes from named, seeded streams, so a run is a function of its scenario and seed; a manifest records the scenario and calibration hashes and `voss repro` regenerates result hashes.

Population and churn. Workers are drawn from three tiers (nightly, idle-hours and dedicated) with heterogeneous hardware, home metros and substrates. Donation sessions follow lognormal, Weibull or nightly-schedule length distributions; a share of departures is graceful (with a drain notice) and the rest abrupt; correlated shocks remove a fraction of a city at once. Three churn regimes (low, medium, high) plus burst and correlated variants parameterize these. No public consumer-donation trace exists, so the regime parameters are tagged *assumed*; their shapes follow volunteer-availability studies [6], and E14 sweeps them.

Workload. The function catalogue has 12 functions modeled on SeBS benchmarks [15] with sync, async and filler SLO

TABLE I
CALIBRATION ENTRIES (S UNLESS NOTED): PLACEHOLDER VERSUS MEASURED ON THE STAND-IN LAPTOP. RATIO IS MEASURED OVER PLACEHOLDER.

Entry	Placeholder	Tag	Measured	Ratio
TPM quote	0.25	literature	0.5062	2.0×
Verifier (check quote + signature)	0.01	assumed	0.03985	4.0×
SHA-256 per MB	0.0007	literature	0.003714	5.3×
Firecracker boot to init (warm cache, HDD)	0.15	literature	0.1542	1.0×
Firecracker snapshot load (warm, HDD)	0.02	literature	0.006767	0.3×
Snapshot create (512 MB, HDD)	0.2	literature	6.68	33.4×
Reboot to kernel start (firmware+loader+kernel)	15	assumed	21.05	1.4×
Kernel start to agent ready	4	assumed	30.74	7.7×

classes. Tenants have a sensitivity weight σ_t. Arrivals follow a Markov-modulated Poisson process (4× bursts) with Zipf-skewed function popularity and a diurnal modulation; a flash-crowd regime multiplies the rate. A sync invocation's deadline is k times its warm latency ($k = 5$ for the sync functions). Unless stated, runs use 15 simulated minutes, a 20 inv/s offered load and 200–600 workers (as stated per experiment); the grids for E18 and E20 differ and are described with their results.

Adversaries. A fraction α of attested workers is *omitting*: it follows the protocol, keeps heartbeats flowing, and silently drops, stalls or throttles work (black-hole, slow, and colluding-defection variants). By the threat-reduction argument (Sec. III-C) no adversary on an attested substrate forges results or reads secrets in the attested substrates; the `none` substrate, used for unattested baselines, admits both.

B. Calibration

Seed parameters in the specification are literature values or assumptions, tagged as such and never silently promoted. Measured values come only from calibration bundles. We measured a bundle on one consumer machine, an Intel Core i3-7020U laptop with a discrete TPM 2.0 and a hard disk, which stands in for the target node. The harness reboots the machine into the appliance and records wall-clock timestamps (resolution about 1 s); microbenchmarks cover TPM quotes, hashing, Firecracker boot and snapshots. Table I compares the measured values with the placeholders used before measurement. The most consequential differences are slower reboot-to-kernel (21 s against 15 s), a much slower kernel-to-agent phase (31 s) and snapshot creation on a hard disk (6.7 s for 512 MB). Reboot to a fully attested worker takes 44 s (median of 9 boots). Firecracker boot with a warm page cache is 154 ms, close to the literature figure [7]; with a cold cache it rises to 1.45 s. E24 re-runs the key comparison under both bundles (Sec. VI-I).

Two limits matter. First, the machine is not the intended donor node and lacks AMD SEV-SNP and Intel TDX, so *confidential-VM and DRTM substrate parameters are literature values* drawn from recent measurement studies [17]–[19]. Second, the appliance image was exercised in host mode (the TPM measurement chain and quote path on the host, microVMs through Firecracker) rather than as a dedicated locked image on bare metal, so end-to-end session establishment (44 s) is a measured sum, not a measured attestation of the final image.

C. Protocol

Each experiment is a frozen specification of base scenario, systems and a grid; *every* cell is run with seeds 1–10 (three seeds for the two largest scalability points, whose runs take up to 20 wall-clock minutes and 9 GB). All comparisons are *paired*: for a metric m, system A and baseline B, we form $\Delta_s = m_A(s) - m_B(s)$ per seed s within the same scenario cell and report the mean with a 95% percentile-bootstrap interval of the paired differences [33]. Relative changes are Δ/m_B with m_B the baseline mean; interval ends can therefore exceed 100%. Tables report mean ± the half-width of the bootstrap interval of the mean.

Each hypothesis is stated before the run together with a verdict rule applied mechanically to the intervals: *supported* when the interval excludes zero in the predicted direction and the mean effect is at least a stated relative margin, *weak* when it excludes zero but the effect is below the margin, *inconclusive* when it includes zero, and *refuted* when it excludes zero in the opposite direction. The scorecard (Sec. VI-J) lists all of them. Specifications were frozen before the final sweep, and the deviations from the pilots are recorded in a decision log. The manuscript's numbers, tables and several sentence-level claims are produced by a script that reads the result tables and *fails* if a result stops supporting the sentence it backs.

D. Metrics and viability

We report success (fraction of invocations completed), the deadline-met fraction, sync P50/P99 end-to-end latency, *waste* (extra accepted attempts per successful invocation: retries, hedges and adopted re-dispatches), *donor memory* (GB held warm), and *key-holder exposure* $E = \sum_t \sigma_t \sum_j (1 - a_j)\, y_{tj}$ at the end of the run (Sec. III). Exposure is comparable only within a substrate mix, because it weights each key-holding host by $(1 - a_j)$, the residual lack of assurance of its substrate.

A system is *viable* in a scenario cell if the lower 95% bound of its success is at least 0.99 and either the lower bound of the deadline-met fraction is at least the stated target or its sync P99 is below an absolute target (2, 3 or 5 s). E20 sweeps three two-dimensional panels (churn × adversary fraction; substrate × load; population × load) at 300 workers, 10 simulated minutes, comparing VOSS, the strongest simple baseline, and the same baseline without attestation. The viability map is thus a statement about *where* each design works, not an average.

E. Baselines

Baselines use the same simulator, candidate sets and cost matrix as VOSS and differ only in the registered policy. *No planning* (never pre-warms), *keep-alive TTL* (an idle timer), *greedy planner* (the same objective solved by a heuristic), *heartbeat-only* detection, *non-robust* planning ($f = 0$), and the *simple baseline*: least-loaded placement on the lowest-latency attested worker, no planning and no hedging. Commercial-style references (*Lambda-like*, *Azure-like*) are always-on datacenter hosts in the router metros with per-function keep-alive (10 and 20 min) and nearest-region routing, priced from list prices; they run on the same engine so that differences are due to the policy and substrate, not the simulator.

TABLE II
SIMULATED SESSION ESTABLISHMENT AND START LATENCY (S) FROM EACH WARM-STATE LEVEL (L0 NOTHING CACHED, L3 WARM IDLE VM; "–" NOT APPLICABLE). PRE-MEASUREMENT PARAMETERS (TABLE I).

Substrate	Session	L0	L1	L2	L3
appliance (TPM)	19.2	1.45	0.346	0.0285	0.00197
DRTM hypervisor	2.31	1.44	0.34	0.0284	0.00196
CVM (SEV-SNP)	1.41	2.31	1.19	0.155	0.002
CoCo pool	8.11	2.25	1.14	–	0.002
no attestation	1.06	1.45	0.344	0.0278	0.00201

VI. RESULTS

Unless stated, differences are paired by seed over 10 seeds with 95% bootstrap intervals; the complete list of hypotheses and verdicts is Table XII. Where a result contradicts a design claim we say so in the section where it appears.

A. Trust establishment and cold start (E-CS1)

Table II gives, per substrate, the simulated time to establish a donation session and the start latency from each level of the warm-state ladder. Two facts shape everything that follows. First, the appliance pays a large *one-off* cost per session (19 s with the pre-measurement parameters; 44 s measured end to end on the laptop), against 1.4 s for a confidential VM and 8.1 s for a confidential-container pool: trust on the appliance is expensive to establish and everything is lost when the session ends. Second, once the session exists the appliance is as fast as an unattested Firecracker host: warm and snapshot starts are within a few percent of the unattested figures (H5, supported), because attestation is paid per session rather than per invocation. Confidential VMs invert the trade: cheap sessions but slower lower ladder levels (Table II). Our a-priori expectation that confidential-VM classes establish trust 10–100× *slower* than the appliance is refuted: they are about 93% faster, because the appliance's cost is a reboot. Measurements on the laptop agree with the simulated ladder: a warm Firecracker start takes 154 ms, a cold-cache boot 1.45 s, and creating a 512 MB snapshot 6.7 s on a hard disk, which makes snapshots something to plan rather than to take on demand.

B. Where is the system viable? (E20)

Averages hide the regimes in which a design fails, so we map viability (Sec. V) over three two-dimensional panels. Figure 3 colors each cell by which system is viable under the default criterion (success ≥ 0.99 and deadline-met ≥ 0.95) and prints VOSS's P99. Table III counts VOSS-viable cells under other targets.

VOSS is the *only* viable system in all 9 cells of panel A, which spans three churn regimes and up to 20% omitting owners, in 9 of 16 cells of panel B and in 4 of 12 cells of panel C. With an absolute sync P99 target of 3 s it is viable in 9 of 9, 13 of 16 and 10 of 12 cells, and the simple baseline in none of them; at 5 s VOSS is viable in 9, 16 and 10 cells. The two criteria measure different things (a deadline is a multiple of the warm latency; the absolute target is what a user perceives), which is why a baseline can pass one and fail the other.

TABLE III
NUMBER OF CELLS IN WHICH VOSS IS VIABLE, BY TARGET (DEADLINE-MET FRACTION, OR ABSOLUTE SYNC P99).

Panel	0.90	0.95	0.98	2 s	3 s	5 s	Cells
	deadline-met target			sync P99 target			
A: churn × α	9	9	0	8	9	9	9
B: substrate × load	16	11	0	6	13	16	16
C: N × load	8	8	0	8	10	10	12

TABLE IV
PLANNER COMPARISON (E04): MEAN ± 95% HALF-WIDTH OVER 10 SEEDS. MET: DEADLINE-MET FRACTION; WARM: GB OF IDLE WARM MEMORY.

Regime	Planner	P99 (s)	Met	Warm GB	Exposure	Solve (s)
churn_med	no planning	2.78±1.35	0.940±0.017	1±0	8.5±1.4	0.003±0.000
churn_med	keep-alive TTL	1.78±0.48	0.971±0.018	61±9	7.4±1.1	0.003±0.001
churn_med	greedy planner	1.29±0.09	0.978±0.015	94±14	19.9±1.8	0.010±0.001
churn_med	robust MILP	1.26±0.11	0.979±0.014	131±19	20.1±1.7	5.627±0.689
churn_high	no planning	3.48±1.32	0.923±0.029	1±0	7.5±1.2	0.002±0.000
churn_high	keep-alive TTL	3.29±1.92	0.940±0.030	59±10	7.4±1.0	0.002±0.000
churn_high	greedy planner	1.42±0.17	0.958±0.028	87±13	15.8±1.2	0.010±0.002
churn_high	robust MILP	1.38±0.16	0.959±0.031	115±12	15.0±0.8	5.473±0.817

The map also shows where VOSS is *not* the right choice. On confidential-VM and confidential-container substrates at higher load, the simple baseline is the only viable system in 4 cells (Fig. 3b): we conjecture that with cheap session establishment there is little cold-start cost for planning to remove, while it still adds exposure and memory; we did not test this. With 100 workers no system is viable in any cell of panel C (4 cells), so the design needs a minimum population, and at a deadline-met target of 0.98 *no* system is viable in any cell: the "98% of sync invocations within 5× warm latency" service level is out of reach in this setting for VOSS and the baselines alike.

C. Planning trust and warm state (E04, E19)

Table IV compares planners on the same scenarios under medium and high churn. Planning is the largest single lever: the robust MILP cuts sync P99 by 55% (medium churn) and 60% (high churn) against no planning, and by 29% and 58% against a keep-alive TTL (H1, supported). It pays with donor memory: about 2.1× the warm memory of keep-alive at medium churn. The comparison that motivates the integer program is weaker. The MILP is only 2.5% better than the greedy planner at high churn (H8, interval includes zero, inconclusive) while its solve time is 5.5 s per cluster against 10 ms. We cannot, on this evidence, claim that the exact solver pays for itself. What it gives, beyond latency, is the coverage guarantee of Eq. (2), whose value is shown below.

Figure 4 removes one component of VOSS at a time (E19). Removing lifecycle-aware placement costs 3.3 s of sync P99, removing planning 1.1 s (and lowers exposure by 31, because less is pre-warmed and fewer hosts hold keys), removing the robust constraint 0.8 s (exposure 24 lower) and removing hedging 0.3 s. Geo-clustering is the only component whose removal *improves* P99 (0.09 s lower), and it also cuts exposure by 34: Sec. VI-E. Putting reputation into the cost matrix instead of using it as a tie-breaker raises exposure by 19. The ablations also show where trust and latency trade against each other: the same components that buy P99 (planning, robustness) raise exposure.

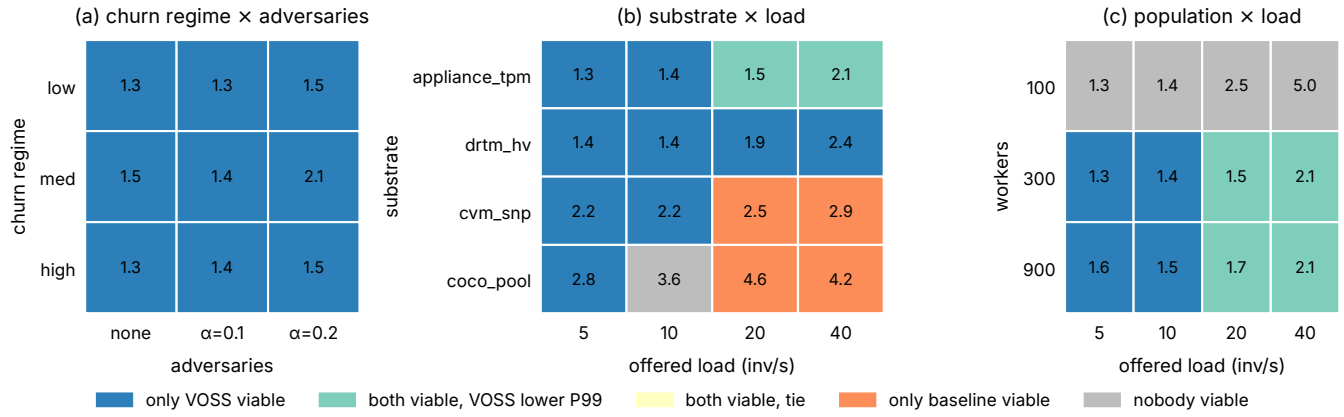


Fig. 3. Viability maps (E20). Cell color: which of VOSS and the best simple baseline is viable (success ≥ 0.99, deadline-met ≥ 0.95); numbers: VOSS sync P99 in seconds. (a) churn regime against adversary fraction, (b) substrate against offered load (inv/s), (c) population against offered load.

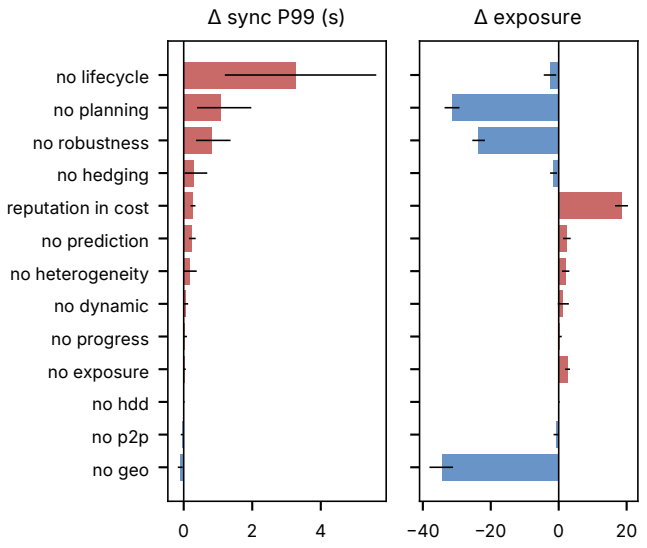


Fig. 4. Ablations (E19): change when a component is removed, relative to full VOSS (paired, 95% intervals). Left: sync P99; right: key-holder exposure.

TABLE V
SCHEDULER FAULTS (E15): TIME TO DETECTION, FAILOVER, SUCCESS AND P99 WITHIN THE FAULT WINDOW.

Fault	Detector	Detect (s)	Failover (s)	Success (%)	P99 in window (s)
crash	progress	2.2±0.3	2.7±0.4	99.95±0.02	1.28±0.15
crash	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.66±0.33
partition	progress	2.2±0.3	2.6±0.4	99.95±0.02	1.28±0.16
partition	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.67±0.31
blackhole	progress	4.2±3.0	4.6±3.0	99.95±0.02	1.28±0.15
blackhole	heartbeat_only	60.0±0.0	59.3±1.1	96.81±0.51	2.90±0.08
slow	progress	13.9±7.9	14.3±8.2	99.95±0.03	1.34±0.14
slow	heartbeat_only	60.0±0.0	59.3±1.1	99.55±0.41	2.00±0.33

TABLE VI
OMITTING OWNERS (E08), MEAN ± 95% HALF-WIDTH OVER 10 SEEDS. OMISSIONS: RESULTS DROPPED BY ADVERSARIES.

Adversaries	System	Success (%)	P99 (s)	Waste (%)	Omissions
none α=0	VOSS	99.99±0.01	1.27±0.11	1.7±0.5	0±0
none α=0	VOSS, f=0	99.99±0.01	1.60±0.24	2.2±0.6	0±0
none α=0	heartbeat-only	99.98±0.01	1.36±0.15	2.0±0.7	0±0
mixed α=0.1	VOSS	99.95±0.04	1.32±0.12	2.2±0.5	38±14
mixed α=0.1	VOSS, f=0	99.96±0.03	2.15±0.60	2.9±0.6	37±11
mixed α=0.1	heartbeat-only	98.19±0.75	1.36±0.13	4.0±0.9	298±115
mixed α=0.2	VOSS	99.95±0.03	1.43±0.16	2.6±0.5	61±13
mixed α=0.2	VOSS, f=0	99.83±0.17	2.05±0.52	3.5±0.8	80±31
mixed α=0.2	heartbeat-only	96.79±1.13	1.39±0.16	5.6±1.3	527±161
all α=0.2	VOSS	99.94±0.05	1.37±0.10	2.3±0.7	31±9
all α=0.2	VOSS, f=0	99.96±0.03	2.28±1.00	2.7±0.5	28±8
all α=0.2	heartbeat-only	97.70±1.26	1.42±0.14	5.2±2.4	385±227

D. Faults, adversaries and the latency–exposure trade-off (E08, E15, E16, E23)

Detection. Table V injects crashes, partitions, black-holing and slowed scheduler hosts. Heartbeats alone cannot see a black hole: the host keeps sending liveness while dropping work, so detection waits for the 60 s timeout (60 s) and the run suffers 132 timeouts, against 4.2 s and 0.7 with progress-based detection (H-FT1, supported). For plain crashes progress detection needs 2.2 s and failover 2.7 s. A slowed-but-live scheduler is the hardest case: failover takes 14 s on average because progress evidence must accumulate before the slowdown is distinguishable from load.

Omitting owners. Table VI sweeps the fraction α of omitting owners. With $\alpha = 0.2$ VOSS keeps success at 99.95%, whereas heartbeat-only detection with otherwise identical machinery drops to 96.8% (H4, supported). Progress detection also gives 9× fewer results dropped by omitting owners at $\alpha = 0.2$. The hypothesis that the *robust* planning constraint is what keeps success is not supported: setting $f = 0$ changes success by only 0.11 percentage points (H2, inconclusive), though it lengthens P99 (Table VI). Detection and re-dispatch, not redundant coverage, carry the success; robust planning buys tail latency.

Hedging. Hedging duplicates risky invocations. Always hedging cuts P99 but wastes 10.6% extra attempts. Risk-proportional hedging (hedge with probability tied to estimated risk, within a 5% budget) wastes 1.8%, and its P99 is 13% lower with $\alpha = 0.1$ but with an interval from -32% to +1% (H-FT3, inconclusive). It is cheap insurance, not demonstrated to be necessary.

Pareto front. Robustness f and the exposure weight trade

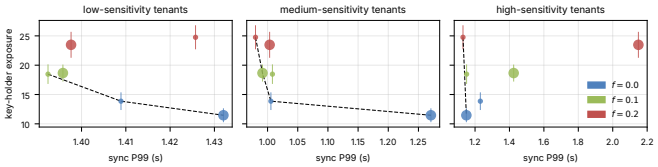


Fig. 5. Latency–exposure–robustness (E23). Each point is a (f, exposure weight) configuration (marker size: weight) with 95% error bars on exposure; the dashed line is the Pareto front per tenant sensitivity class.

TABLE VII
GEO-CLUSTERING (E05): MEAN ± 95% HALF-WIDTH OVER 10 SEEDS.

Population	Clustering	P99 (s)	Network (s)	Clusters	Exposure
Europe	one global scheduler	1.07±0.09	0.092±0.015	1.0±0.0	12±1
Europe	static regions	1.07±0.09	0.092±0.015	1.0±0.0	12±1
Europe	static k-means	1.25±0.09	0.124±0.031	10.4±0.5	63±4
Europe	dynamic	1.24±0.10	0.127±0.037	9.1±0.6	61±4
EU + NA	one global scheduler	1.14±0.09	0.154±0.021	1.0±0.0	14±1
EU + NA	static regions	1.21±0.15	0.124±0.015	2.0±0.0	24±2
EU + NA	static k-means	1.31±0.13	0.151±0.031	7.5±0.4	54±3
EU + NA	dynamic	1.33±0.20	0.127±0.013	6.4±0.6	52±4
world	one global scheduler	1.21±0.11	0.192±0.016	1.0±0.0	13±1
world	static regions	1.36±0.12	0.167±0.013	6.6±0.3	43±4
world	static k-means	1.31±0.11	0.164±0.020	7.0±0.5	48±3
world	dynamic	1.27±0.11	0.158±0.018	6.0±0.3	46±3

latency against key-holder exposure (E23, Fig. 5). The aggregate front has 3 points. Exposure grows with f (13, 19 and 24 at $f = 0$, 0.1, 0.2), consistent with covering any f failures requiring more key-holding hosts. Aggregate sync P99 stays within 1.28–1.33 s across configurations, so the knee at $f = 0.1$ saves 25% of exposure against the best-latency front point for 0.5% more P99 (H3, supported): moderate exposure budgets are cheap. Per tenant class the picture is less benign. Low-sensitivity tenants stay at about 1.4 s or below, but for high-sensitivity tenants P99 reaches 2.1 s in one robust configuration, while an $f = 0$ configuration has the lowest exposure at nearly the best P99. The aggregate knee therefore should not be applied to every class; sensitivity classes need their own operating point.

E. Geography, routing and caching (E05–E07, E09)

The geographic hierarchy, a central idea of the design, does not pay off at the scale we simulate. Table VII compares a single global scheduler against static regions, static k-means and dynamic clusters. Dynamic clusters have *higher* P99 than the global scheduler by 17% (Europe), 17% (EU+NA) and 5% (world) (H7, refuted at all three populations), and hold 5.1× the key-holder exposure in Europe, since each of the 9 clusters must cover demand with its own key holders. The network share of latency does fall in some populations (Table VII), but not enough to offset the cost of splitting the pool. We attribute this to fewer hosts per cluster for pooling warm state and keys, but did not isolate the mechanism. Our results show no population in which geo-clustering wins at the scale we simulate.

Routing matters more than clustering. Without flash crowds, nearest-region routing meets 0.991 of deadlines (P99 1.20 s), whereas routing to the least-loaded cluster meets 0.911 (P99 1.39 s) because it ignores distance. Spilling to other clusters is rare and priced (E07). For caching (E09) the cost-aware GDSF policy with peer-to-peer fetch and an HDD tier is at most 3% better than LRU in sync P99 and the intervals include zero (H6, inconclusive). All variants start about 98% of invocations from a warm VM, which leaves little for the disk tiers to improve.

TABLE VIII
SIMPLE BASELINE VERSUS VOSS (E10).

Hardware	System	P99 (s)	Met	Exposure
homogeneous	simple baseline	3.88±0.98	0.962±0.012	64±4
homogeneous	VOSS	2.38±0.47	0.978±0.007	122±7
mixed	simple baseline	2.66±0.24	0.969±0.012	54±3
mixed	VOSS	1.31±0.09	0.989±0.003	95±4

TABLE IX
COMMERCIAL-STYLE REFERENCES VERSUS VOSS (E17). MET: DEADLINE-MET FRACTION; COST IN USD PER MILLION INVOCATIONS.

Load (inv/s)	System	P50 (s)	P99 (s)	Met	USD/M inv.
10	VOSS	0.425±0.083	1.33±0.08	0.987±0.005	19.6±0.9
10	Lambda-like	0.292±0.090	1.21±0.12	0.999±0.000	12.1±4.5
10	Azure-like	0.292±0.090	1.30±0.13	0.997±0.000	12.0±4.4
20	VOSS	0.425±0.082	1.27±0.11	0.987±0.006	9.8±0.5
20	Lambda-like	0.292±0.090	1.11±0.10	0.999±0.000	12.1±4.6
20	Azure-like	0.292±0.090	1.17±0.11	0.998±0.000	11.9±4.4
40	VOSS	0.442±0.084	1.50±0.23	0.979±0.009	4.9±0.2
40	Lambda-like	0.292±0.091	1.10±0.10	0.999±0.000	12.1±4.6
40	Azure-like	0.292±0.091	1.14±0.11	0.998±0.000	11.9±4.5

F. Is the machinery needed? Simple baseline and commercial references (E10, E17)

Table VIII compares VOSS with the simple baseline (least-loaded placement on the nearest attested worker, no planning). We expected the baseline to match VOSS on homogeneous, stable, benign hardware. It does not: VOSS has 63% lower P99 (H11, refuted), even though the pool is homogeneous; we attribute this to planning's pre-warming against churn, which the baseline lacks. The baseline, however, holds about half of VOSS's exposure (VOSS is 1.9× higher) consistent with its placing keys on fewer hosts. Whether the latency is worth the exposure is a policy choice, and the E23 front (above) lets a deployer pick.

Table IX compares VOSS with commercial-style references on the same workload and engine, priced with list prices. At 20 inv/s VOSS's P99 is 14% and its P50 45% above the Lambda-like reference, which uses always-on datacenter hosts. We attribute the gap to perishable sandboxes and longer network paths on volunteer hardware. Cost per million invocations, including the orchestrator tier but excluding volunteer credits, falls from 20 USD at 10 inv/s through 10 to 5 at 40 inv/s as the fixed tier is amortized, against about 12 USD for the references. VOSS is thus cost-competitive only above roughly 20 inv/s on this workload, and it never has lower latency.

G. Accounting and reputation (E12, E13)

Attested receipts are off the critical path (H10, supported): lengthening the settlement period from 10 to 300 s or the batch from 0.2 to 5 s changes P99 by no more than ±1.5%. The stream is small, about 17 rows/s and 4.5 KB/s at 20 inv/s offered load, every receipt is accounted for (0 unexplained receipts, at most 1 receipt missing at the end of a run) and verification

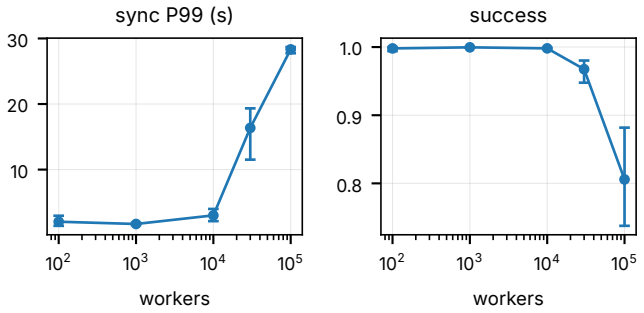


Fig. 6. Scalability (E18): sync P99 and success against population (log scale), mean with 95% intervals; the two largest points use 3 seeds.

TABLE X
SCALABILITY (E18): ORCHESTRATOR AND CONTROL-PLANE LOAD.

Workers	Seeds	Success (%)	P99 (s)	Decide P99 (s)	Msg/s/worker	Orch. CPU (s/s)
100	10	99.8±0.3	2.1±0.8	0.10±0.01	0.97±0.04	0.000±0.000
1 000	10	100.0±0.1	1.7±0.2	0.10±0.01	0.74±0.01	0.002±0.000
10 000	10	99.8±0.1	3.0±0.9	0.48±0.59	0.73±0.01	0.021±0.000
30 000	3	96.8±1.6	16.4±3.9	14.45±11.95	0.74±0.01	0.062±0.000
100 000	3	80.6±7.2	28.3±0.5	29.83±8.05	0.82±0.03	0.207±0.001

costs 0.8 ms per second. The settlement period sets the latency from receipt to credit: 6, 30 and 118 s for 10, 60 and 300 s periods. Reputation, however, adds little. Using a Beta-decay score only as a tie-breaker lowers P99 by 2.4% (E12, weak), and credit is highly concentrated: the Gini coefficient is 0.93, the top 10% of workers earn 93% of credit and 65% earn almost none, which is a fairness problem for an incentive that relies on credits.

H. Scalability (E18)

Figure 6 and Table X scale the population from 10^2 to 10^5 workers at constant load per worker (0.01 inv/s) in a cheaper "scalability mode" (coarse heartbeats, greedy planning, 2 simulated minutes). Up to 10^4 workers success is 99.8%. At 3×10^4 it falls to 97% with a decision P99 of 14 s, and at 10^5 to 81% with sync P99 28 s and decision P99 30 s: decision latency grows from about 0.1 s to tens of seconds. Per-worker control traffic stays flat (0.70–1.08 messages per second per worker), so the limit lies in the central decision path rather than in the message volume; we did not isolate which orchestrator or router component saturates first, and the two largest points use three seeds. We read this as a measured limit of the current orchestrator tier, not of the approach.

I. Sensitivity to calibration (E24)

Do conclusions depend on the measured values? We repeated the key comparison under the placeholder and the laptop-measured bundles (Table XI). VOSS's P99 is 1.17 s with measured parameters against 1.32 s with placeholders, but the simple baseline moves from 3.9 s to 4.6 s: the advantage of planning *grows* under the measured parameters. This shows that the main qualitative conclusion is not an artifact of the placeholders; absolute values should not be read as predictions.

TABLE XI
CALIBRATION SENSITIVITY (E24): PLACEHOLDER VERSUS LAPTOP-MEASURED PARAMETERS.

Bundle	System	P50 (s)	P99 (s)	Met	Exposure
placeholder (C0 design)	VOSS	0.429±0.083	1.32±0.11	0.984±0.006	47±4
placeholder (C0 design)	simple baseline	0.509±0.095	3.89±0.60	0.965±0.010	53±3
measured (laptop)	VOSS	0.239±0.040	1.17±0.25	0.978±0.005	37±4
measured (laptop)	simple baseline	0.266±0.052	4.58±0.65	0.954±0.011	53±3

J. Scorecard

Of the hypotheses stated before the runs, 11 are supported, 2 weak, 6 inconclusive and 5 refuted (Table XII). The refuted and inconclusive entries are as informative as the supported ones: geo-clustering (H7), the simple baseline on homogeneous hosts (H11), and the trust-establishment cost ratio of confidential VMs (H5) were wrong in the direction predicted, and the robust constraint, the MILP over greedy planning, caching and hedging are inconclusive.

VII. DISCUSSION

What the evidence supports. The central design claim holds in simulation: treating trust and warm state as a planned, perishable resource beats reactive policies. Planning cuts tail latency by half or more against no planning under churn, the lifecycle-aware placer and the planner are the two largest contributors in the ablation, and progress-based detection is what keeps success high when owners omit. These are statements about mechanisms and are supported by paired intervals. They hold across churn regimes and adversary fractions in the viability maps (Sec. VI-B), not only at one operating point.

What it does not. Four expected benefits did not materialize. (i) Geo-clustering is slower than a single scheduler and holds several times the exposure. (ii) The exact MILP is not demonstrably better than a greedy planner, so the solver's cost (5.5 s per cluster) is justified only by the coverage guarantee, which we could not show matters for *success* (H2). (iii) Caching sophistication and reputation tie-breaking change P99 by a few percent at most, and credit under the current rules is extremely concentrated. (iv) A commercial-style reference has lower latency, and VOSS is cheaper per invocation only once the fixed orchestrator tier is amortized over enough load. None of this contradicts the premise that volunteer hardware can serve latency-sensitive functions at some SLO; it narrows which components earn their complexity. A simpler VOSS without geo-clustering, reputation and tiered caches is the version our evidence favors.

Exposure is the price. Most latency gains are bought with exposure: planning, robustness and warm state all enlarge the set of hosts that hold a tenant's key. The simple baseline has about half of VOSS's exposure, and the Pareto front shows that moderate robustness captures most of the benefit. We deliberately report exposure as a separate axis; a deployer with high-sensitivity tenants would pick a lower f and a higher exposure weight, accepting the latency cost shown in Fig. 5.

Why the attestation argument matters. Every robustness result above uses omission and timing faults only. If attestation

TABLE XII
HYPOTHESIS SCORECARD. Δ REL.: PAIRED RELATIVE CHANGE WITH 95% BOOTSTRAP INTERVAL; VERDICT BY THE RULES OF SEC. V. "–": COST-SIDE INFORMATION, NO VERDICT.

H	Claim	Exp.	Cell	Δ rel. [95% CI]	Verdict
H1	MILP planner cuts sync P99 vs no planning	E04	churn_med	-54.6% [-105.6, -13.6]	**supported**
H1	MILP planner cuts sync P99 vs no planning	E04	churn_high	-60.2% [-96.8, -26.1]	**supported**
H1	... and vs keep-alive TTL	E04	churn_med	-28.9% [-57.6, -8.3]	**supported**
H1	... and vs keep-alive TTL	E04	churn_high	-57.9% [-117.5, -10.9]	**supported**
H1	donor memory held warm: MILP vs keep-alive TTL (cost side)	E04	churn_med	+112.3% [+90.7, +129.4]	–
H2	f-robust plans keep success where non-robust collapse (α=0.2 mixed)	E08	mixed α=0.2	-0.1% [-0.3, +0.0]	inconclusive
H3	Pareto front has a knee; moderate exposure loses little latency	E23		+0.5%	**supported**
H4	progress detection beats heartbeat-only on success under α=0.2	E08	mixed α=0.2	-3.2% [-4.3, -2.1]	**supported**
H5	appliance warm start $\approx$ Firecracker without attestation (L3)	E-CS1 (sim)		-2.0%	**supported**
H5	appliance snapshot start $\approx$ Firecracker without attestation (L2)	E-CS1 (sim)		+2.5%	**supported**
H5	CVM classes are 10-100x slower than the appliance to establish trust	E-CS1 (sim)		-92.7%	**refuted**
H6	GDSF + P2P + HDD tier beat LRU without P2P/HDD (sync P99)	E09		+3.1% [-2.7, +12.3]	inconclusive
H6	... GDSF vs LRU alone	E09		+1.8% [-0.1, +4.2]	inconclusive
H7	geo clusters reduce sync P99 vs one global scheduler (EU)	E05	regions=[eu]	+16.5% [+10.8, +21.6]	**refuted**
H7	... EU+NA	E05	regions=[eu, na]	+16.6% [+5.2, +33.1]	**refuted**
H7	... world	E05	regions=[]	+4.7% [+1.2, +8.8]	**refuted**
H8	planner MILP beats greedy by $\geq$5% sync P99 (high churn)	E04	churn_high	-2.5% [-5.8, +0.5]	inconclusive
H9	distribution-aware (quantile) beats expectation-only on P99 at high churn	E03	churn_high, name=blr	-2.8% [-5.3, -0.8]	weak
H10	accounting is off the critical path: settlement period 10 s vs 300 s	E13	faults=[]	+1.5% [-0.2, +4.2]	**supported**
H10	... receipt batch 0.2 s vs 5 s	E13	faults=[]	-1.0% [-3.3, +1.0]	**supported**
H11	simple baseline matches VOSS when homogeneous, stable and benign	E10	hw=C1	+62.8% [+25.0, +111.5]	**refuted**
H-FT1	progress detection vs heartbeat-only: success with a black-holed scheduler	E15	faults=sched_blackhole	+3.2% [+2.7, +3.7]	**supported**
H-FT3	risk-proportional hedging (θ=.3, 5%) cuts sync P99 under α=0.1	E16	mixed α=0.1	-13.3% [-32.5, +1.4]	inconclusive
H-FT3	... at what extra-attempt cost	E16	mixed α=0.1	+253.1% [+182.9, +332.3]	–
RQ15	VOSS vs Lambda-like reference, sync P99 at 20 inv/s	E17	rate=20	+14.2% [+10.9, +18.1]	–
E12	reputation tie-break (beta-decay) improves sync P99 vs no reputation	E12		-2.4% [-4.4, -0.8]	weak
E12	... and success rate	E12		-0.0% [-0.0, +0.0]	inconclusive

failed, for example through the physical attacks excluded in Sec. III-C, adversaries could forge results and read keys, and exposure would no longer be a proxy for risk. The design's honesty is conditional on attestation holding, and we therefore treat liveness and owner notices as hints and verify progress.

A. Threats to validity

- **Simulation only.** No multi-machine deployment has validated the simulator end to end. Policies are the real code, but the world model (network, churn, adversaries) is a model. We therefore interpret differences between designs, not absolute latencies.
- **Calibration scope.** Calibration comes from one consumer laptop with a hard disk. Confidential-VM and DRTM parameters are literature values, and the laptop-measured appliance runs in host mode. E24 shows the main comparison survives a change of bundle, but a different CPU, SSD or firmware could shift the absolute costs.
- **Assumed workload and churn.** No public trace of consumer donation sessions exists; the churn shapes are assumed from availability studies and swept in E14. The function catalogue follows SeBS but its sizes are assumed. The adversary model is omission and timing only, with a fixed menu of strategies. An adaptive adversary that learns the scheduler could do worse.
- **Scale.** The two largest scalability points use three seeds and a cheaper simulator mode, and a 10^5-worker deployment may behave differently.
- **Statistics and selection.** Ten seeds give wide intervals for tail metrics; several comparisons are inconclusive rather than null. We ran many experiments, and a verdict on a single comparison should be read with that in mind. Specifications were frozen before the final sweep, but pilots informed which cells to include (recorded in the decision log).
- **Bibliography and prior numbers.** Literature parameters (for example confidential-VM startup) are quoted from the cited studies and are tagged for re-verification.

B. Future work

Validate on a small real testbed with end-to-end measurements (the first experiment in our plan, not yet run); measure confidential-VM hosts to replace literature parameters; shard the orchestrator tier and re-measure scalability; revisit geo-

clustering at larger populations and with shared trust across clusters; and study adaptive adversaries and incentives given the observed credit concentration.

VIII. Conclusion

Attestation turns malicious volunteer owners into omission and timing faults, and what remains is the scheduling of a perishable resource: trust. A two-timescale design, a robust slow-loop plan over trust and warm state plus a fast cost-based placer with progress-based failure detection, is viable in simulation in regimes where simple baselines are not (9 of 9 and 13 of 16 cells at a 3 s sync P99 target), cuts tail latency by 55–60% against no planning, and keeps success at 99.95% with 20% omitting owners. The advantage is not universal: on confidential-VM substrates at high load the simple baseline is the only viable system on the deadline-met criterion. Our paired, mechanically judged evaluation also shows what does not pay: geo-clustering, the exact solver over greedy planning, caching sophistication and reputation tie-breaking, and a commercial-style reference remains faster. The design is better read as a small core (attestation, planned trust and warm state, progress detection) than as the full catalog of mechanisms we explored. The next step is a real multi-machine testbed to validate the simulator, which this paper does not yet provide.

References

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX ATC*, 2020, pp. 205–218.

[2] D. P. Anderson, J. Cobb, E. Korpela, M. Lebofsky, and D. Werthimer, "SETI@home: An experiment in public-resource computing," *Commun. ACM*, vol. 45, no. 11, pp. 56–61, 2002.

[3] D. P. Anderson, "BOINC: A system for public-resource computing and storage," in *Proc. IEEE/ACM Int. Workshop on Grid Computing*, 2004, pp. 4–10.

[4] D. Levin, J. R. Douceur, J. R. Lorch, and T. Moscibroda, "TrInc: Small trusted hardware for large distributed systems," in *Proc. USENIX NSDI*, 2009, pp. 1–14.

[5] G. S. Veronese, M. Correia, A. N. Bessani, L. C. Lung, and P. Verissimo, "Efficient Byzantine fault-tolerance," *IEEE Trans. Computers*, vol. 62, no. 1, pp. 16–30, 2013.

[6] B. Javadi, D. Kondo, A. Iosup, and D. Epema, "The Failure Trace Archive: Enabling the comparison of failure measurements and models of distributed systems," *J. Parallel and Distributed Computing*, vol. 73, no. 8, pp. 1208–1223, 2013.

[7] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *Proc. USENIX NSDI*, 2020, pp. 419–434.

[8] J. Cadden, T. Unger, Y. Awad, H. Dong, O. Krieger, and J. Appavoo, "SEUSS: Skip redundant paths to make serverless fast," in *Proc. EuroSys*, 2020, pp. 32:1–32:15.

[9] L. Ao, G. Porter, and G. M. Voelker, "FaaSnap: FaaS made fast using snapshot-based VMs," in *Proc. EuroSys*, 2022, pp. 730–746.

[10] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "SOCK: Rapid task provisioning with serverless-optimized containers," in *Proc. USENIX ATC*, 2018, pp. 57–70.

[11] A. Fuerst and P. Sharma, "FaaSCache: Keeping serverless computing alive with greedy-dual caching," in *Proc. ACM ASPLOS*, 2021, pp. 386–400.

[12] Y. Zhang, Í. Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delimitrou, and R. Bianchini, "Faster and cheaper serverless computing on harvested resources," in *Proc. ACM SOSP*, 2021, pp. 724–739.

[13] T. Rausch, A. Rashed, and S. Dustdar, "Optimized container scheduling for data-intensive serverless edge computing," *Future Generation Computer Systems*, vol. 114, pp. 259–271, 2021, tODO verify.

[14] P. Cao and S. Irani, "Cost-aware WWW proxy caching algorithms," in *Proc. USENIX Symp. on Internet Technologies and Systems*, 1997, pp. 193–206.

[15] M. Copik, G. Kwasniewski, M. Besta, M. Podstawski, and T. Hoefler, "SeBS: A serverless benchmark suite for function-as-a-service computing," in *Proc. ACM/IFIP Middleware*, 2021, pp. 64–78.

[16] AMD, "SEV-SNP: Strengthening VM isolation with integrity protection and more," White paper, 2020.

[17] M. Misono, D. Stavrakakis, N. Santos, and P. Bhatotia, "Confidential VMs explained: An empirical analysis of AMD SEV-SNP and Intel TDX," in *Proc. ACM SIGMETRICS*, 2025, tODO verify.

[18] D. Li, Z. Mi, C. Ji, Y. Tan, B. Zang, H. Guan, and H. Chen, "Bifrost: Analysis and optimization of network I/O tax in confidential virtual machines," in *Proc. USENIX ATC*, 2023, tODO verify.

[19] B. Holmes, J. Waterman, and D. Williams, "SEVeriFast: Minimizing the root of trust for fast startup of SEV microvms," in *Proc. ACM ASPLOS*, 2024, tODO verify title and pages.

[20] C. Segarra, T. Pfandzelter *et al.*, "Serverless confidential containers: Challenges and opportunities," in *Proc. Workshop on Serverless Systems, Applications and Methodology (SESAME)*, 2024, tODO verify authors.

[21] "Confidential serverless computing (Wallet)," arXiv:2504.21518; USENIX NSDI 2026, 2025, tODO verify authors and final title.

[22] H. N. Jacob, C. Werling, R. Buhren, and J.-P. Seifert, "faulTPM: Exposing AMD fTPMs' deepest secrets," in *Proc. IEEE European Symp. on Security and Privacy (EuroS&P)*, 2023, tODO verify.

[23] N. Schear, P. T. Cable, T. M. Moyer, B. Richard, and R. Rudd, "Bootstrapping and maintaining trust in the cloud," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2016, pp. 65–77.

[24] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[25] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, "Sparrow: Distributed, low latency scheduling," in *Proc. ACM SOSP*, 2013, pp. 69–84.

[26] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. EuroSys*, 2015, pp. 18:1–18:17.

[27] D. Bertsimas and M. Sim, "The price of robustness," *Operations Research*, vol. 52, no. 1, pp. 35–53, 2004.

[28] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[29] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, "The EigenTrust algorithm for reputation management in P2P networks," in *Proc. Int. World Wide Web Conf. (WWW)*, 2003, pp. 640–651.

[30] J. R. Douceur, "The Sybil attack," in *Proc. Int. Workshop on Peer-to-Peer Systems (IPTPS)*, 2002, pp. 251–260.

[31] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ accrual failure detector," in *Proc. IEEE Symp. on Reliable Distributed Systems (SRDS)*, 2004, pp. 66–78.

[32] G. Shafer and V. Vovk, "A tutorial on conformal prediction," *J. Machine Learning Research*, vol. 9, pp. 371–421, 2008.

[33] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.